

BMS Auth Module

Mukammal Code Explanation — Roman Urdu Mein

Yeh document aapke 5 PHP files ki line-by-line aur function-by-function explanation hai. Har file ka code, uski har line ka matlab, aur har function ka kaam Roman Urdu mein samjhaya gaya hai. Team lead ke sawal ke liye taiyaar raho!

#	File	Kaam (Purpose)
1	google-auth.php	Google login URL banata hai aur user ko redirect karta hai
2	google-callback.php	Google se wapas aata hai, user data save / session set karta hai
3	index.php	Login form HTML — email/password + Google button
4	logout.php	Session destroy karke index.php pe bhejta hai
5	process.php	Email/password verify karke session set karta hai

FILE 1 — google-auth.php

Is File Ka Maqsad (Purpose)

Yeh file sirf EK kaam karti hai: Google OAuth login ka URL banati hai aur user ko uss URL pe redirect karti hai taake user apni Google account se login kar sake.

Code:

```
require_once '../vendor/autoload.php';
```

Explanation:

require_once — yeh PHP ka function hai jo ek file ko exactly ek baar include karta hai. Agar file nahi mili toh fatal error aata hai.

vendor/autoload.php — Composer (PHP package manager) ka generated file hai. Jab bhi hum composer se koi library install karte hain (jaise Google Client), toh Composer ek autoload.php file banata hai. Is file ko include karne se saari libraries automatically available ho jaati hain bina manually include kiye.

Code:

```
$client = new Google\Client();
```

Explanation:

new keyword — PHP mein naya object banana ke liye use hota hai.

Google\Client() — Google API PHP Client Library ka main class hai. Yeh class Google ke saath baat karne ka kaam karti hai (auth, APIs, tokens sab kuch).

\$client — variable mein Google Client object store ho gaya.

Code:

```
$client->setClientId('209930267043-...');
```

Explanation:

setClientId() — Google Client ka method hai jo Client ID set karta hai.

Client ID kya hai? — Google Cloud Console pe jaake project banate hain, wahan se OAuth 2.0 credentials milte hain. Client ID app ki pehchaan hai (jaise ID card). Iss se Google jaanta hai kaunsi app login maang rahi hai.

-> (arrow operator) — PHP mein object ka method ya property access karne ke liye use hota hai.

Code:

```
$client->setClientSecret('GOCSPX-...');
```

Explanation:

setClientSecret() — Client Secret set karta hai.

Client Secret kya hai? — Yeh ek private password jaisa hai jo sirf aapki app aur Google ke darmiyan hota hai. KABHI bhi public nahi karna chahiye (GitHub pe upload na karo!). Yeh confirm karta hai ke request genuine app se aa rahi hai.

Code:

```
$client->setRedirectUri('http://localhost/BMS-BETA/modules/auth/google-callback.php');
```

Explanation:

setRedirectUri() — Redirect URI set karta hai.

Redirect URI kya hai? — Jab user Google pe successfully login kare, Google user ko wapas is URL pe bhejta hai. Yeh URI Google Cloud Console mein pehle se register honi chahiye warna Google error dega.

Code:

```
$client->addScope('email');  
$client->addScope('profile');
```

Explanation:

addScope() — Yeh method batata hai ke hum Google se user ki kaunsi kaunsi information access karna chahte hain.

'email' scope — user ki email address access karne ki permission.

'profile' scope — user ka naam, profile picture, Google ID access karne ki permission.

Scope = Permission. Jitna scope maango, Google utni hi cheez share kare ga.

Code:

```
$auth_url = $client->createAuthUrl();
```

Explanation:

createAuthUrl() — Google Client ka method hai jo ek complete Google OAuth URL banata hai. Iss URL mein client_id, redirect_uri, scopes, aur security parameters automatically add ho jaate hain. Result \$auth_url variable mein store hota hai.

Code:

```
header('Location: ' . filter_var($auth_url, FILTER_SANITIZE_URL));
```

Explanation:

header() — PHP ka built-in function hai jo HTTP headers bhejta hai browser ko.

'Location: ...' — yeh ek redirect header hai. Browser isko dekh ke automatically us URL pe chala jaata hai.

filter_var() — PHP ka built-in function jo data ko filter/sanitize karta hai.

FILTER_SANITIZE_URL — yeh ek constant hai jo URL se unsafe characters remove karta hai (security ke liye).

. (dot) — PHP mein string concatenation operator hai (do strings jodna).

IMPORTANT: Client Secret ko .env file mein rakhna chahiye, source code mein hardcode karna security risk hai!

FILE 2 — google-callback.php

Is File Ka Maqsad (Purpose)

Jab user Google pe login karta hai, Google user ko wapas is file pe bhejta hai ek 'code' parameter ke saath. Yeh file us code ko use karke: 1. Access token leta hai 2. User ki info fetch karta hai 3. Database mein check/save karta hai 4. Session set karta hai 5. Redirect karta hai

Code:

```
include('../../includes/header-min.php');
```

Explanation:

include() — PHP function jo doosri PHP file ka content current file mein inject karta hai.

header-min.php — yeh file typically database connection (\$link), session_start(), aur helper functions (fetch_data, add_data etc.) load karti hai. include() mein agar file nahi mili toh warning aata hai (require ki tarah fatal error nahi).

Code:

```
if (isset($_GET['code'])) {
```

Explanation:

isset() — PHP ka built-in function jo check karta hai ke variable set hai ya nahi (null nahi).

\$_GET — PHP ka superglobal array hai jo URL parameters store karta hai.

Yahan check ho raha hai ke Google ne URL mein 'code' parameter bheja hai ya nahi. Google OAuth hamesha success pe 'code' parameter bhejta hai.

Code:

```
$token = $client->fetchAccessTokenWithAuthCode($_GET['code']);
```

Explanation:

fetchAccessTokenWithAuthCode() — Google Client ka method hai.

Kaam: Google ke authorization 'code' ko Access Token mein convert karta hai.

Access Token kya hota hai? — Ek temporary key jis se hum Google APIs call kar sakte hain. Yeh ek short-lived token hota hai (usually 1 hour valid).

Result \$token array mein aata hai jisme access_token, refresh_token wagara hote hain.

Code:

```
$client->setAccessToken($token['access_token']);
```

Explanation:

setAccessToken() — Google Client ko batata hai ke ab is Access Token se requests karo.

\$token['access_token'] — token array se sirf access_token ki value nikali.

Code:

```
$oauth = new Google\Service\Oauth2($client);
```

Explanation:

Google\Service\Oauth2 — Google ki Oauth2 service class hai.

Yeh class user ki basic info (email, name, id) fetch karne ke liye use hoti hai.

Constructor mein \$client pass kiya gaya hai taake authenticated requests ho sakein.

Code:

```
$userInfo = $oauth->userinfo->get();
```

Explanation:

userinfo->get() — Google Oauth2 API call karta hai aur logged-in user ki info return karta hai.

\$userInfo ek object hota hai jisme email, name, id, picture wagara hoti hai.

Code:

```
$google_id = $userInfo->getId();  
$email = $userInfo->getEmail();  
$first_name = $userInfo->getGivenName();  
$last_name = $userInfo->getFamilyName();
```

Explanation:

getId() — User ka unique Google ID return karta hai (numeric string).

getEmail() — User ki email address return karta hai.

getGivenName() — User ka first name return karta hai.

getFamilyName() — User ka last/family name return karta hai.

Yeh sab Google's Oauth2 UserInfo object ke getter methods hain.

Code:

```
$check_query = "SELECT * FROM ums_users WHERE email = '$email'";  
$existingUser = fetch_data($link, $check_query);
```

Explanation:

SELECT * — database se saari columns fetch karo.

WHERE email = '\$email' — sirf woh row jahan email match kare.

fetch_data() — yeh custom helper function hai (probably header-min.php mein defined) jo SQL query run karta hai aur results array mein return karta hai.

\$link — database connection variable hai.

Code:

```
if (!empty($existingUser)) {
```

Explanation:

empty() — PHP built-in function jo check karta hai ke variable empty hai ya nahi.

! (not operator) — opposite karta hai, toh !empty() matlab 'agar empty nahi hai'.

Yahan check: agar user pehle se database mein hai toh is block mein jao.

Code:

```
if (empty($user['google_id'])) {
    update_data($link, 'ums_users', ['google_id' => $google_id], ['user_id' =>
    $user['user_id']]);
}
```

Explanation:

Agar existing user ka google_id database mein nahi tha (pehle email/password se register kiya tha), toh uska google_id update karo.

update_data() — custom helper function jo SQL UPDATE query run karta hai.

Parameters: connection, table name, jo update karna hai (array), where condition (array).

Code:

```
$userData = ['email' => $email, 'google_id' => $google_id, ...];
$newUserId = add_data($link, 'ums_users', $userData);
```

Explanation:

Agar user bilkul naya hai (pehle kabhi register nahi kiya) toh nayi entry banao.

add_data() — custom helper function jo SQL INSERT query run karta hai.

Return value: naye inserted record ka auto-increment ID.

'role_id' => 2 — naye Google users ko default role 2 (Employee) assign kiya.

'status' => 1 — account active hai.

Code:

```
$emp_check = fetch_data($link, "SELECT emp_id FROM tbl_employees WHERE email = '$email'
LIMIT 1");
$emp_id = !empty($emp_check) ? $emp_check[0]['emp_id'] : null;
```

Explanation:

Yeh code tbl_employees table mein check karta hai ke koi employee record hai is email se.

LIMIT 1 — sirf pehla result lo, performance ke liye.

Ternary operator (?:) — chhota if-else hai:

!empty(\$emp_check) ? \$emp_check[0]['emp_id'] : null

Matlab: agar result mila toh emp_id lo, warna null.

Yeh isliye kiya ke BMS mein ums_users (login users) aur tbl_employees (HR data) alag tables hain.

Code:

```
$_SESSION['user_id'] = $user['user_id'];
$_SESSION['user_name'] = $user['first_name'] . ' ' . $user['last_name'];
$_SESSION['role_id'] = $user['role_id'];
$_SESSION['superAdmin'] = ($user['role_id'] == 1) ? 1 : 0;
$_SESSION['emp_id'] = $emp_id;
```

Explanation:

\$_SESSION — PHP ka superglobal array hai jo server pe session data store karta hai.

Session kya hota hai? — Server side storage jahan user ka data temporarily save hota hai jab tak browser open hai. Session ID cookie mein store hoti hai.

user_id, user_name, role_id — baad mein har page pe check karne ke liye.

superAdmin — ternary: agar role_id == 1 toh superAdmin = 1, warna 0. Permissions check ke liye use hota hai.

emp_id — employee specific data access ke liye.

Code:

```
if ($user['role_id'] == 1) {
    header('Location: ../../index.php');
} else {
    header('Location: ../../modules/profile-page.php');
}
```

Explanation:

Role based redirect — agar superAdmin (role_id=1) hai toh main dashboard pe bhejo, warna profile page pe.

header('Location: ...') — HTTP redirect karta hai.

exit() — script execution yahan rok do, warna PHP code aage bhi chalega.

FILE 3 — index.php (Login Form)

Is File Ka Maqsad (Purpose)

Yeh main login page hai. Isme HTML form hai (email + password), Google login button, aur error message display hota hai agar login fail hoi ho.

Code:

```
$base_url = '/BMS-BETA';
```

Explanation:

Ek variable mein base URL store kiya gaya hai taake assets (CSS, JS, images) ke paths mein baar baar /BMS-BETA likhna na pare. DRY principle (Don't Repeat Yourself).

Code:

```
session_start();
```

Explanation:

session_start() — PHP ka built-in function jo session initialize karta hai.

Har page pe jahan session use karna ho, sabse pehle yeh call karna zaroori hai.

Agar already session chal raha hai toh woh resume karta hai.

Code:

```
$error_msg = isset($_SESSION['login_error']) ? $_SESSION['login_error'] : '';  
unset($_SESSION['login_error']);
```

Explanation:

Agar process.php ne login fail pe session mein error message rakha tha, toh use \$error_msg mein store karo.

unset() — variable ya array key ko delete karta hai. Yahan session se error delete kiya taake refresh pe dobara na dikhe (Post/Redirect/Get pattern).

Ternary operator: isset check karke ya toh value lo ya empty string.

Code:

```
data-layout='vertical' data-topbar='light' data-sidebar='dark' ...
```

Explanation:

Yeh HTML data attributes hain jo JavaScript theme system use karta hai.

data-* attributes — apni custom data HTML elements pe store karne ka HTML5 standard tareeqa.

JavaScript in attributes ko read karke layout change karta hai (sidebar color, topbar style wagera).

Code:

```
...
```

Explanation:

PHP ke andar if condition — agar \$error_msg empty nahi hai toh error div show karo.
Yeh PHP ka alternative if syntax hai (colon aur endif) jo HTML ke saath mix karne mein zyada readable hota hai.
alert alert-danger — Bootstrap CSS classes hain jo red error box dikhati hain.

Code:

Explanation:

action='process.php' — form submit hone pe data process.php pe jayega.
method='POST' — HTTP POST method use hoga.
POST vs GET: POST mein data URL mein nahi dikhta (password ke liye zaroori), GET mein URL mein dikhta hai.

Code:

Explanation:

type='email' — browser automatically email format validate karta hai.
name='email' — form submit hone pe PHP mein \$_POST['email'] se access hoga.
required — HTML5 attribute, browser empty submit nahi karne deta.

Code:

Explanation:

type='password' — input field ke characters dots mein dikhai dete hain (security).
name='password' — PHP mein \$_POST['password'] se milega.

Code:

```
onclick="window.location.href='google-auth.php'"
```

Explanation:

onclick — JavaScript event handler jo button click pe chalata hai.
window.location.href — browser ko nayi URL pe redirect karta hai.
Iss se user google-auth.php pe jata hai jo Google login shuru karta hai.

FILE 4 — logout.php

Is File Ka Maqsad (Purpose)

User ko properly logout karta hai — session destroy karta hai, cookies clear karta hai, aur login page pe redirect karta hai.

Code:

```
if (session_status() === PHP_SESSION_NONE) { session_start(); }
```

Explanation:

session_status() — current session ki state return karta hai.
PHP_SESSION_NONE — constant hai jiska matlab session abhi start nahi hui.
PHP_SESSION_ACTIVE — session chal rahi hai.

Yeh check isliye hai ke agar session already start hai toh dobara session_start() na call ho (warning aata hai). Safe coding practice hai.

Code:

```
$_SESSION = array();
```

Explanation:

\$_SESSION ko empty array assign karna — iska matlab saara session data delete ho gaya.

array() — PHP mein empty array banane ka tareeqa (ya [] bhi likh sakte hain).

Yeh session server data clear karta hai memory se.

Code:

```
if (ini_get('session.use_cookies')) {  
    $params = session_get_cookie_params();  
    setcookie(session_name(), '', time() - 42000, ...);  
}
```

Explanation:

ini_get('session.use_cookies') — PHP configuration check karta hai ke sessions cookies use kar rahi hain ya nahi.

session_get_cookie_params() — session cookie ki settings return karta hai (path, domain, secure, httponly).

setcookie() — browser mein cookie set karta hai.

session_name() — session cookie ka naam (usually 'PHPSESSID').

" — empty value (cookie clear karna).

time() - 42000 — past time set karna (42000 seconds pehle). Browser expired cookies delete kar deta hai. Yeh session cookie delete karne ka tareeqa hai.

Code:

```
session_destroy();
```

Explanation:

session_destroy() — server pe stored session data permanently delete karta hai.

\$_SESSION = array() ne memory se data clear kiya, session_destroy() ne server pe file/database se delete kiya.

Dono zaroori hain proper logout ke liye.

Code:

```
header('Location: index.php');  
exit();
```

Explanation:

Logout ke baad user ko login page pe redirect karo.

exit() — PHP script execution yahan rok do. Yeh zaroori hai kyunke header() ke baad bhi PHP code chalta rehta hai agar exit() na ho.

FILE 5 — process.php (Login Logic)

Is File Ka Maqsad (Purpose)

Yeh file login form ka backend hai. Email/password verify karta hai, CSRF attack se protect karta hai, aur successful login pe session set karta hai.

Code:

```
if (!isset($_POST['csrf_token']) || !validate_csrf_token($_POST['csrf_token'])) {
```

Explanation:

CSRF kya hota hai? — Cross-Site Request Forgery — ek attack jisme dusri website aapki taraf se request bhejti hai bina aapki marzi ke.

CSRF Token — ek random secret value jo form mein hidden field mein hoti hai. Server check karta hai ke yeh token valid hai ya nahi.

isset(\$_POST['csrf_token']) — check ke token form mein aaya?

validate_csrf_token() — custom function jo token verify karta hai.

|| (OR operator) — agar dono mein se koi bhi true hai toh block mein jao.

! (NOT) — opposite karna. Toh: 'agar token nahi aaya OR invalid hai toh error do.'

Code:

```
$email = clean_string($link, $_POST['email']);  
$password = $_POST['password'];
```

Explanation:

clean_string() — custom helper function jo email ko database ke liye safe banata hai (SQL injection se bachata hai).

\$_POST['email'] — form se aaya email value.

\$password — password ko clean nahi kiya kyunke password_verify() use hoga jo plaintext compare karta hai hashed password se.

Code:

```
$query = "SELECT u.*, r.role_name  
FROM ums_users u  
LEFT JOIN ums_roles r ON u.role_id = r.role_id  
WHERE u.email = '$email' AND u.status = 1";
```

Explanation:

SELECT u.* — ums_users table ki saari columns lo, u alias hai.

r.role_name — ums_roles table se role_name bhi lo, r alias hai.

LEFT JOIN — dono tables ko join karo role_id se match karke. LEFT JOIN matlab agar role nahi mila toh bhi user row aayegi (null values ke saath).

WHERE u.email = '\$email' — sirf is email wala user.

AND u.status = 1 — sirf active users (disabled accounts login nahi kar sakein).

Code:

```
if (!empty($userData) && password_verify($password, $userData[0]['password'])) {
```

Explanation:

!empty(\$userData) — database mein user mila.

&& (AND operator) — dono conditions true honni chahiyein.

password_verify() — PHP ka built-in function.

Parameter 1: user ne jo password enter kiya (plaintext).

Parameter 2: database mein stored hashed password.

Kaam: plaintext ko hash karta hai aur stored hash se compare karta hai.

Return: true ya false.

Kyun hash? — Passwords kabhi plaintext store nahi karte, agar database leak ho toh passwords safe rahein.

Code:

```
session_regenerate_id(true);
```

Explanation:

session_regenerate_id() — login ke baad naya session ID generate karta hai.

true parameter — purana session destroy kar do.

Kyun zaroori? — Session Fixation attack se bachata hai. Attacker login se pehle session ID jaanta tha, ab naya ID ban gaya toh useless ho gaya.

Code:

```
$_SESSION['superAdmin'] = ($user['role_id'] == 1) ? 1 : 0;  
$_SESSION['type'] = 'admin';
```

Explanation:

Ternary operator: role_id == 1 hai toh superAdmin = 1, warna 0.

\$_SESSION['type'] = 'admin' — yeh manually set kiya (shayad Google login se differentiate karne ke liye, wahan type set nahi tha). Baad mein check ho sakta hai ke user ne kis tarah login kiya.

Code:

```
if ($user['role_id'] == 1) {  
    header('Location: ../../index.php');  
} else {  
    header('Location: ../../modules/profile-page.php');  
}  
exit;
```

Explanation:

Role based redirect:

role_id == 1 (Super Admin) — main dashboard.

Baaki sab — profile page.

exit; — script yahan band karo (header ke baad zaruri).

Code:

```
$_SESSION['login_error'] = 'Invalid email or password.';  
header('Location: index.php');  
exit;
```

Explanation:

Agar login fail hua:

Session mein error message store karo.

index.php pe redirect karo.

index.php pe yeh session error read hoga aur user ko dikha diya jayega.

Phir unset() se delete ho jata hai (ek baar hi dikhta hai).

Yeh Post/Redirect/Get (PRG) pattern hai — browser back button safe rahta hai.

Quick Reference — Important Functions

Function	Type	Kaam
----------	------	------

require_once()	PHP Built-in	File include karo (ek baar, error pe fatal)
include()	PHP Built-in	File include karo (error pe warning)
session_start()	PHP Built-in	Session shuru/resume karo
session_destroy()	PHP Built-in	Session server se delete karo
session_regenerate_id()	PHP Built-in	Naya session ID banao (security)
isset()	PHP Built-in	Variable set hai? (null nahi)
empty()	PHP Built-in	Variable empty hai? (0, "", null, [])
unset()	PHP Built-in	Variable/key delete karo
header()	PHP Built-in	HTTP header bhejo (redirect bhi)
exit() / die()	PHP Built-in	Script execution rokh do
password_verify()	PHP Built-in	Password aur hash compare karo
filter_var()	PHP Built-in	Data filter/sanitize karo
ini_get()	PHP Built-in	PHP config value paro
setcookie()	PHP Built-in	Browser cookie set karo
time()	PHP Built-in	Current Unix timestamp (seconds)
array()	PHP Built-in	Array banao
new Google\Client()	Google Library	Google API client object banao
setClientId()	Google Library	OAuth Client ID set karo
setClientSecret()	Google Library	OAuth Client Secret set karo
setRedirectUri()	Google Library	Callback URL set karo
addScope()	Google Library	Permission scope add karo
createAuthUrl()	Google Library	Google OAuth URL banao
fetchAccessTokenWithAuthCode()	Google Library	Code ko Access Token mein badlo
setAccessToken()	Google Library	Token set karo client mein
userinfo->get()	Google Library	User ki profile info fetch karo
getId()	Google Library	Google User ID lo
getEmail()	Google Library	Email address lo
getGivenName()	Google Library	First name lo
getFamilyName()	Google Library	Last name lo
fetch_data()	Custom Helper	SELECT query run karo, array return
add_data()	Custom Helper	INSERT query run karo, new ID return
update_data()	Custom Helper	UPDATE query run karo
clean_string()	Custom Helper	String sanitize karo (SQL injection safe)
validate_csrf_token()	Custom Helper	CSRF token verify karo

Auth Flow Summary (Poora Process)

EMAIL/PASSWORD LOGIN FLOW:
User → index.php (form fill karo)

```
→ process.php (POST)
→ CSRF check
→ DB se user fetch
→ password_verify()
→ Session set
→ Role check → redirect
```

GOOGLE LOGIN FLOW:

```
User → index.php (Google button)
→ google-auth.php
→ Google Client setup
→ createAuthUrl()
→ Google OAuth page (user login kare)
→ google-callback.php (Google redirect)
→ fetchAccessTokenWithAuthCode()
→ userinfo->get() (email, name, id)
→ DB mein check: user hai?
→ Hai: google_id update karo agar missing
→ Nahi: naya user banao
→ tbl_employees mein emp_id dhoondh
→ Session set karo
→ Role check → redirect
```

LOGOUT FLOW:

```
User → logout.php
→ $_SESSION = array() (data clear)
→ Cookie expire karo
→ session_destroy()
→ index.php redirect
```

All the best! Team lead ke sawal zaroor aaenge in topics pe: CSRF, SQL Injection, Password Hashing, Session Security, OAuth 2.0 Flow, Role-Based Access.